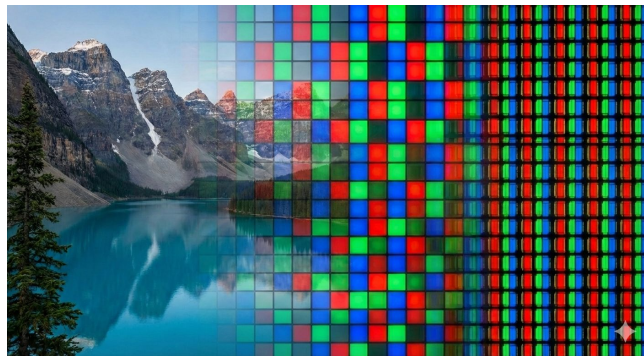


UT4 - Tratamiento de imagen



Jon Apesteguía
2025-2026

Objetivos

- Entender qué es realmente una imagen digital (Muestreo y Cuantificación).
- Visualizar la imagen como una estructura de datos (Matriz).
- Comprender el sistema de coordenadas en computación.
- Tipos de imágenes y profundidad de color.
- Relaciones entre píxeles (Vecindad).

¿Qué es una Imagen Digital?

Definición Matemática: Función $f(x, y)$ donde en cada posición hay un valor de intensidad.

Del Mundo Real al Digital:

- El mundo es continuo (infinitos detalles) → El ordenador es discreto (valores finitos).

Los 2 Procesos de Digitalización:

1. Muestreo (Sampling) → Resolución:

- Decidimos cuántos píxeles tendrá la rejilla.
- *Ejemplo:* 1920x1080 vs 320x240.

2. Cuantificación (Quantization) → Profundidad de Color:

- Decidimos cuántos valores/colores puede tener cada píxel.
- *Ejemplo:* ¿2 colores (Blanco/Negro) o 16 millones (True Color)?

La Unidad Mínima: El Píxel

Origen: "Picture Element". Es el punto más pequeño direccionable.

Atributos:

1. **Posición:** ¿Dónde está? (Fila, Columna).
2. **Valor:** ¿Cómo es?
 - Escala de grises: Un número de brillo (0-255).
 - Color: Un vector de 3 números [R, G, B].

La Imagen como Matriz

Concepto Fundamental: Para el ordenador, **no existe la "foto", solo existen números.**

Representación Matemática: Una imagen es una matriz A de dimensiones $M \times N$.

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,0} & a_{m,1} & \cdots & a_{m,n} \end{bmatrix}$$

Cada celda de la matriz es un píxel.

Sistema de Coordenadas

El error nº1 en Visión Artificial: Confundir Matemáticas con Programación.

Matemáticas (Plano Cartesiano):

- Origen (0,0): Abajo a la izquierda.
- Ejes: x horizontal, y vertical (hacia arriba).

Informática (Matrices/Imágenes):

- **Origen (0,0): Esquina SUPERIOR Izquierda.**
- **Ejes:**
 1. Eje 0 (Vertical hacia abajo) → **FILAS (y)**.
 2. Eje 1 (Horizontal derecha) → **COLUMNAS (x)**.

Regla de Oro en NumPy:

- Accedemos siempre como: `imagen[FILA, COLUMNA]`
- Equivalente a: `imagen[y, x]` (Al revés que en geometría)

Profundidad de Color: ¿Cuántos lápices tenemos?

Concepto: La profundidad de color (Bit Depth) define la cantidad de valores posibles que puede tomar un solo píxel. Es el resultado directo de la **Cuantificación**.

El Estándar: Escala de Grises (uint8)

- En IA y Visión Artificial, la mayoría de algoritmos trabajan en blanco y negro para ahorrar cómputo.
- **Tipo de dato:** uint8 (Unsigned Integer 8-bit).
- **Cálculo:** $2^8 = 256$ niveles de gris.

La Escala:

- **0** → Negro Absoluto (Ausencia de señal).
- **255** → Blanco Puro (Saturación total).
- *Intermedios:* Grises (ej. 128 es gris medio).

El error clásico: Desbordamiento de Bits

El Problema: NumPy optimiza la memoria usando 8 bits. Funciona como un reloj: al llegar al tope, vuelve a empezar.

Ejemplo Práctico:

- Imagina un píxel muy brillante: **Valor 250**.
- Queremos aumentar su brillo sumando **10**.

La Trampa:

- **Matemáticas:** $250 + 10 = 260$ (Blanco saturado).
- **Informática (uint8):** $260 \pmod{256} = 4$ (Negro casi puro).

Consecuencia Visual:

- Al sumar dos imágenes, aparecen manchas negras ("ruido") en las zonas más brillantes.

Solución: Convertir temporalmente a float o int16 para operar, y luego volver a uint8.

Más allá del Gris: Color y Binarias

Imágenes a Color (RGB):

- Ya no es una matriz 2D, es un **Tensor 3D**.
- **Dimensiones:** (Alto, Ancho, 3).
- **El Píxel:** Es un vector [R, G, B].
 - Rojo puro: [255, 0, 0]
 - Amarillo: [255, 255, 0] (Rojo + Verde).
- **Coste:** Ocupa el triple de memoria RAM.

Imágenes Binarias (Máscaras):

- **Valores:** Solo **0** (Fondo) y **1** (Objeto).
- **Uso:** Detección y Segmentación (ej. ¿Hay un coche aquí? Sí/No).

Resumen de tipos

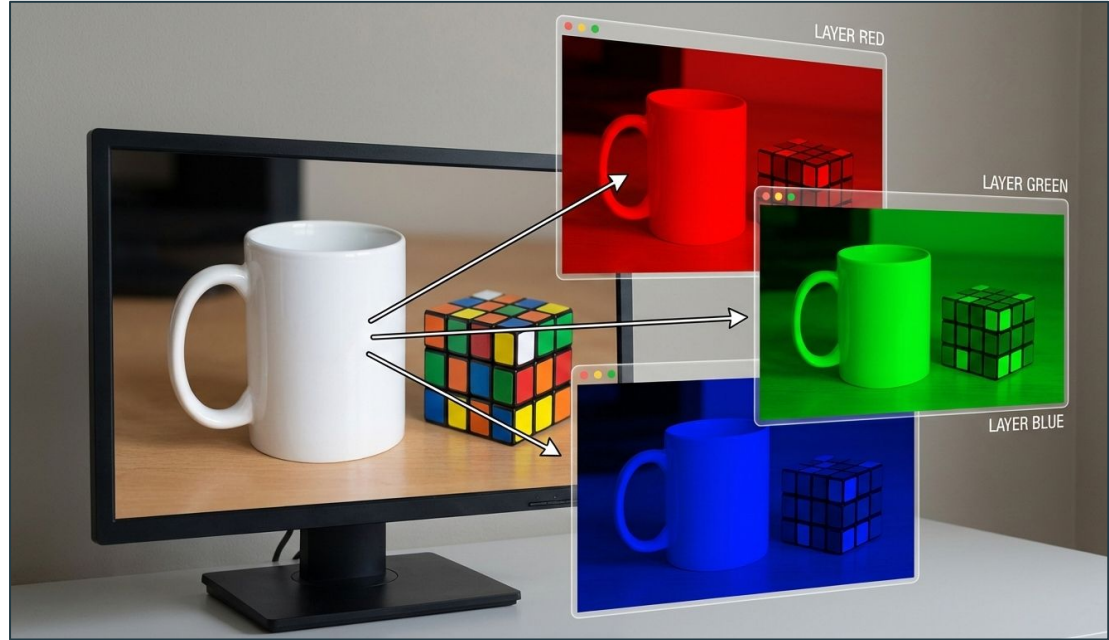
Tipo de Imagen	Canales	Estructura NumPy (.shape)	Rango de Valores	Tipo de Dato (dtype)	Uso Principal
Escala de Grises	1	(Alto, Ancho)	0 a 255	uint8	Procesamiento general, detección de bordes, reconocimiento facial.
Color (RGB)	3	(Alto, Ancho, 3)	0 a 255 (x3)	uint8	Visualización para humanos, segmentación por color.
Binaria	1	(Alto, Ancho)	0 ó 1	bool o uint8	Máscaras, resultado de umbralización, detección de movimiento.
Matemática	1 o 3	(Alto, Ancho, ...)	0.0 a 1.0	float32	Operaciones complejas (divisiones, redes neuronales) para evitar <i>overflow</i> .

Canales de Color

Modelo RGB: Modelo aditivo.

Descomposición:

- Una imagen RGB son **tres matrices de escala de grises apiladas.**



Relaciones entre Píxeles

Importancia: Para procesar una imagen (filtros, bordes), miramos un píxel y a sus vecinos.

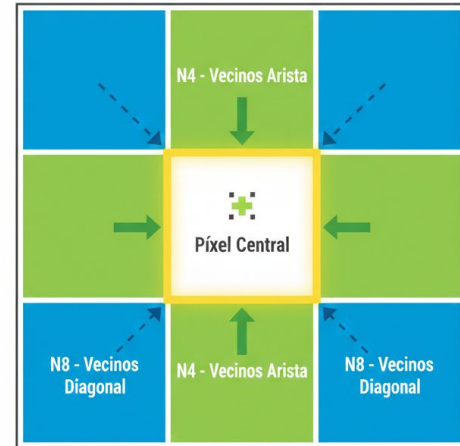
Vecindad-4 (N4):

- Los que comparten una arista (Arriba, Abajo, Izquierda, Derecha).
- Coordenadas: $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$, $(x, y - 1)$

Vecindad-8 (N8):

- Incluye N4 + las 4 diagonales.

Relaciones entre Píxeles: Vecindad N4 y N8



Herramientas de Python

NumPy (np):

- El motor de cálculo.
- La imagen es un array de NumPy.
- Operaciones: Slicing, Suma, Resta.

Matplotlib (plt):

- El visualizador.
- `plt.imread()`: Cargar imagen a array.
- `plt.imshow()`: Pintar array como imagen.

Resumen de Comandos Clave

Cargar: `img = plt.imread('foto.jpg')`

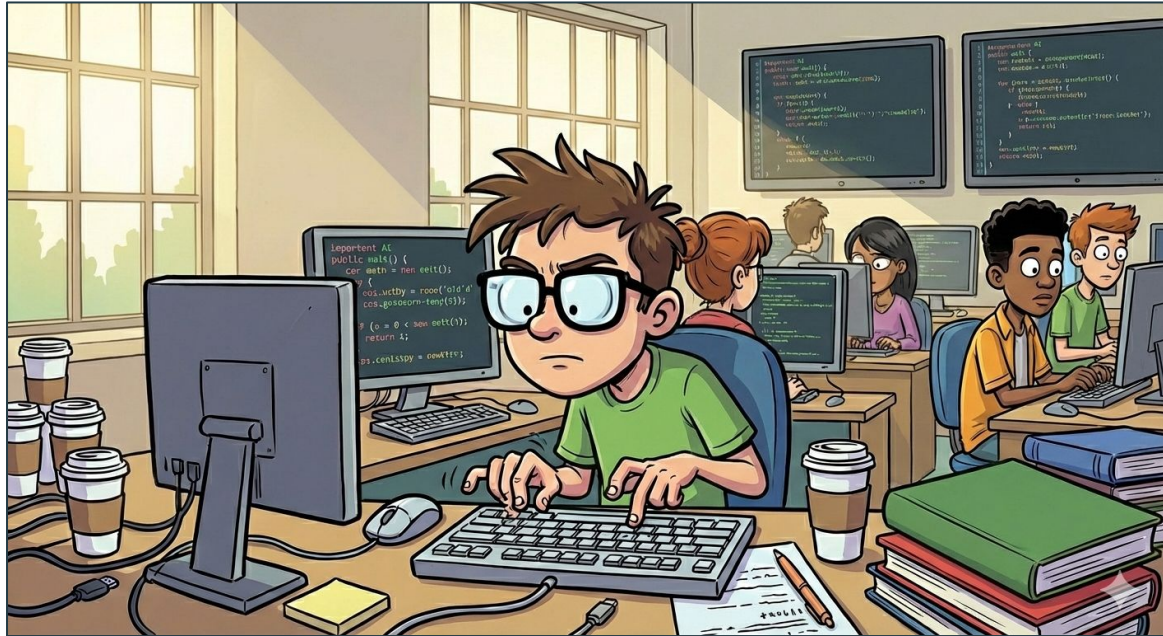
Ver propiedades:

- `img.shape` → (Alto, Ancho, Canales)
- `img.dtype` → `uint8` o `float32`
- `img.min()`, `img.max()` → Rango dinámico

Recortar (Crop): `recorte = img[0:100, 0:100]`

Pasamos a la Práctica

Realiza la Actividad 1: Disección de la Imagen Digital



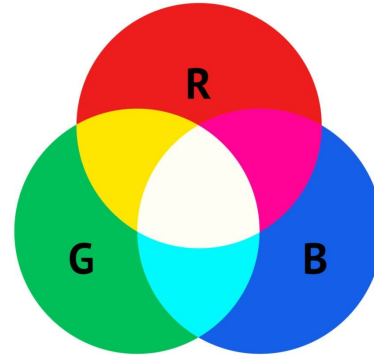
RGB: Bueno para pantallas, malo para algoritmos

Modelo Aditivo: Diseñado para emitir luz (Monitores/Cámaras).

Alta Correlación:

- Si una sombra tapa un objeto rojo, cambian el R, el G y el B a la vez.
- Matemáticamente, "Color" e "Iluminación" están mezclados.

Consecuencia: Es muy difícil programar:
"Detecta objetos rojos independientemente de si es de día o de noche".



R - red

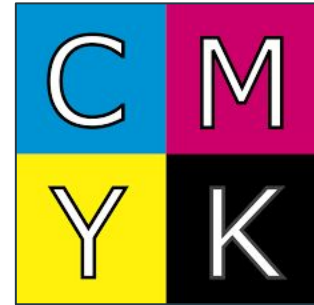
G - green

B - blue

Otros Espacios: CMYK y Escala de Grises

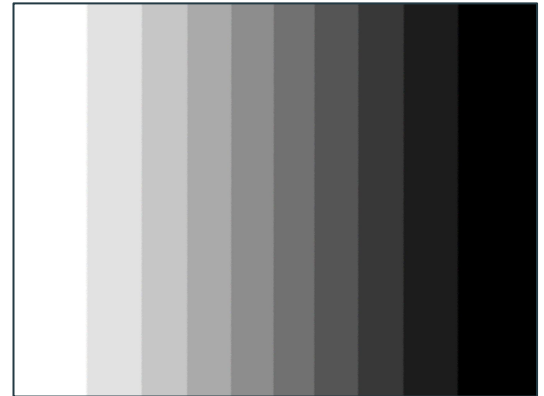
CMYK (Cian, Magenta, Amarillo, Negro):

- Modelo **Sustractivo** (Tintas/Pigmentos).
- Uso exclusivo en impresión. En Visión Artificial **NO** se usa (salvo para OCR de documentos escaneados).



Escala de Grises (1 Canal):

- Elimina la complejidad del color.
- Vital para algoritmos de forma y textura (Detección de bordes, reconocimiento facial).
- Menor coste computacional (1 matriz vs 3 matrices).



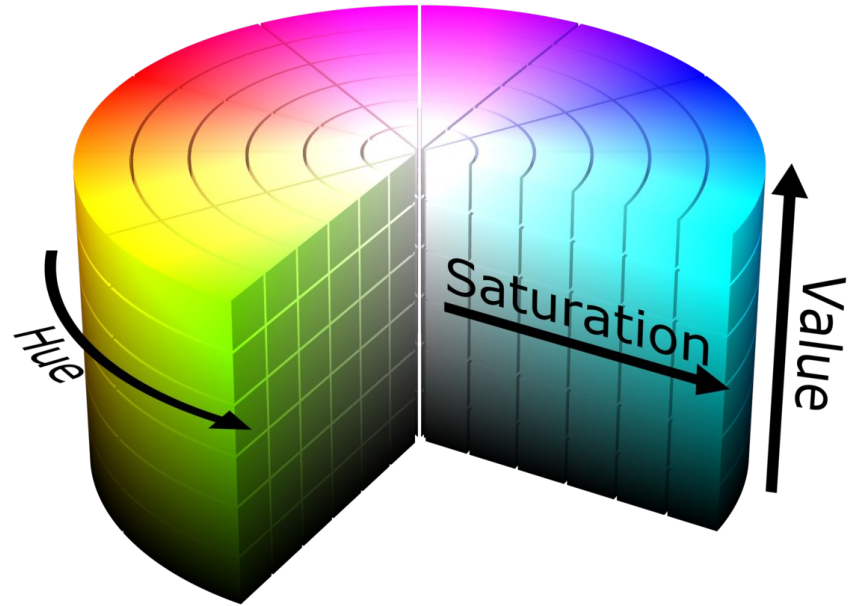
La Solución: Espacio HSV (Hue, Saturation, Value)

Modelo Cilíndrico (Percepción Humana):

- **H (Hue/Matiz):** ¿Qué color es? ($0^\circ - 360^\circ$).
El ADN del color.
- **S (Saturation):** ¿Qué pureza tiene? (0% - 100%).
De gris a vivo.
- **V (Value):** ¿Cuánta luz tiene? (0% - 100%).
De negro a brillante.

La Ventaja en IA:

- Podemos separar la **Crominancia** (Color) de la **Luminancia** (Luz).
- Para seguir una pelota roja, solo miramos el canal **H**, ignorando las sombras en **V**.



El Histograma: El Diagnóstico de la Imagen

Definición: Gráfico de frecuencias que muestra la distribución de la luz en la imagen.

- **Eje X:** Intensidad (0=Negro $\rightarrow$ 255=Blanco).
- **Eje Y:** Cantidad de píxeles.

Utilidad: Permite saber si una imagen es válida para un algoritmo sin necesidad de mirarla.

Regla de Oro: Una buena imagen para IA debe aprovechar todo el rango dinámico.

Patrones de Error: Exposición

Caso A: Subexpuesta (Oscura)

- *Síntoma*: La montaña de datos está pegada a la **izquierda** (0).
- *Pérdida*: Detalles en las sombras (negros empastados).

Caso B: Sobreexpuesta (Quemada)

- *Síntoma*: La montaña está pegada a la **derecha** (255).
- *Pérdida*: Detalles en las luces (blancos rotos). **Irrecuperable**.

Patrones de Error: Contraste

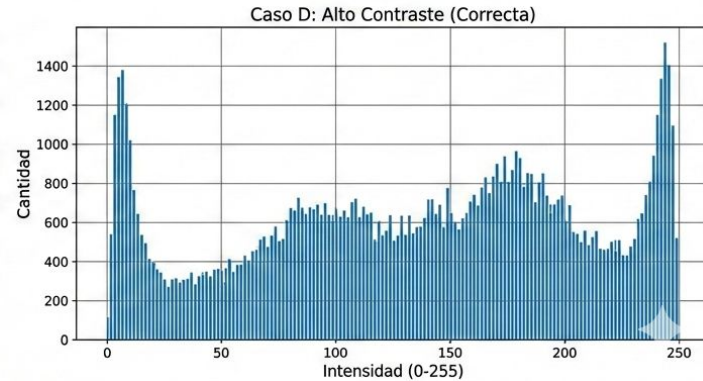
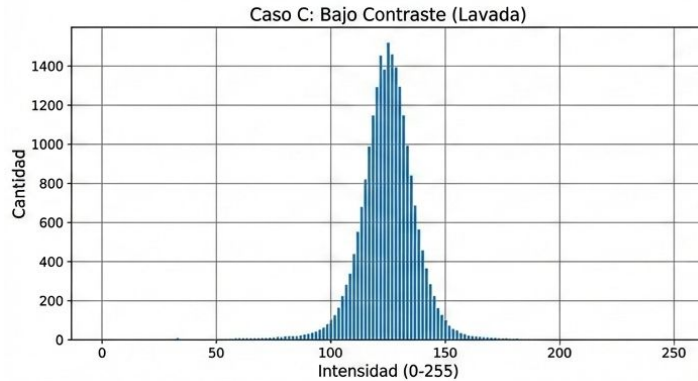
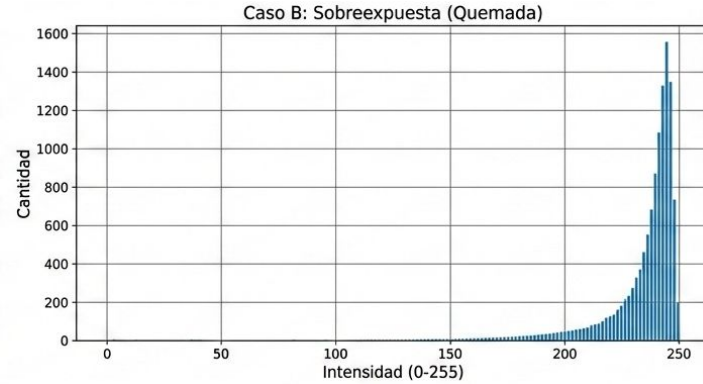
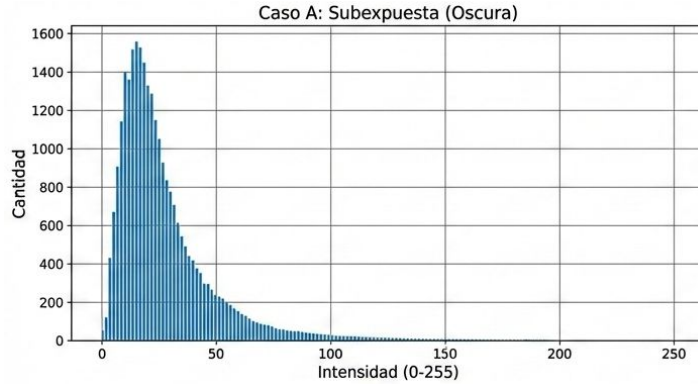
Caso C: Bajo Contraste (Lavada)

- *Síntoma*: Histograma estrecho y centrado.
- *Efecto*: La imagen se ve grisácea. Difícil detectar bordes.

Caso D: Alto Contraste (Correcta)

- *Síntoma*: La distribución se extiende de 0 a 255.
- *Efecto*: Negros profundos y blancos brillantes. Ideal para procesar.

Representación gráfica



Reparación Matemática

Podemos arreglar (algunas) imágenes operando píxel a píxel: $g(x, y) = T[f(x, y)]$

1. **Brillo (Suma):** Desplazamiento horizontal.

- $Píxel_{nuevo} = Píxel_{viejo} + \beta$
- Corrige imágenes oscuras.

2. **Contraste (Multiplicación):** Estiramiento.

- $Píxel_{nuevo} = Píxel_{viejo} \times \alpha$
- $\alpha > 1$: Expande el histograma.

El "Clamping" (Pinzamiento)

El Peligro: Las operaciones matemáticas pueden generar valores fuera del rango uint8 (0-255).

- $200 + 100 = 300 \rightarrow$ Error (Overflow).

La Solución: Función clip.

- Si valor $< 0 \rightarrow$ Se queda en 0.
- Si valor $> 255 \rightarrow$ Se queda en 255.

En Python:

Siempre finalizar las operaciones con esto:

```
imagen_arreglada = np.clip(imagen_calculada, 0, 255).astype('uint8')
```


Del Hardware (RGB) a la Lógica (HSV)

La Realidad del Hardware:

- Las cámaras capturan en **RGB**.
- Las pantallas muestran en **RGB**.
- La imagen "nace" y "muere" en RGB.

La Estrategia del Software:

- El ordenador no "ve" en HSV por arte de magia.
- **Paso Intermedio:** Nosotros convertimos la matriz para trabajar.
- **Analogía:** Es como traducir un texto a otro idioma para buscar palabras clave más fácilmente, y luego volver a traducir el resultado.

¿Qué cambia en la Memoria?

La matriz **NO** cambia de tamaño.

- RGB: (Alto, Ancho, 3)
- HSV: (Alto, Ancho, 3)

Lo que cambia es el significado de la 3ª dimensión (profundidad).

Canal	En Matriz RGB	En Matriz HSV
0	Rojo (Intensidad)	Matiz (Color/Ángulo)
1	Verde (Intensidad)	Saturación (Pureza)
2	Azul (Intensidad)	Valor (Brillo)

El Algoritmo de Detección

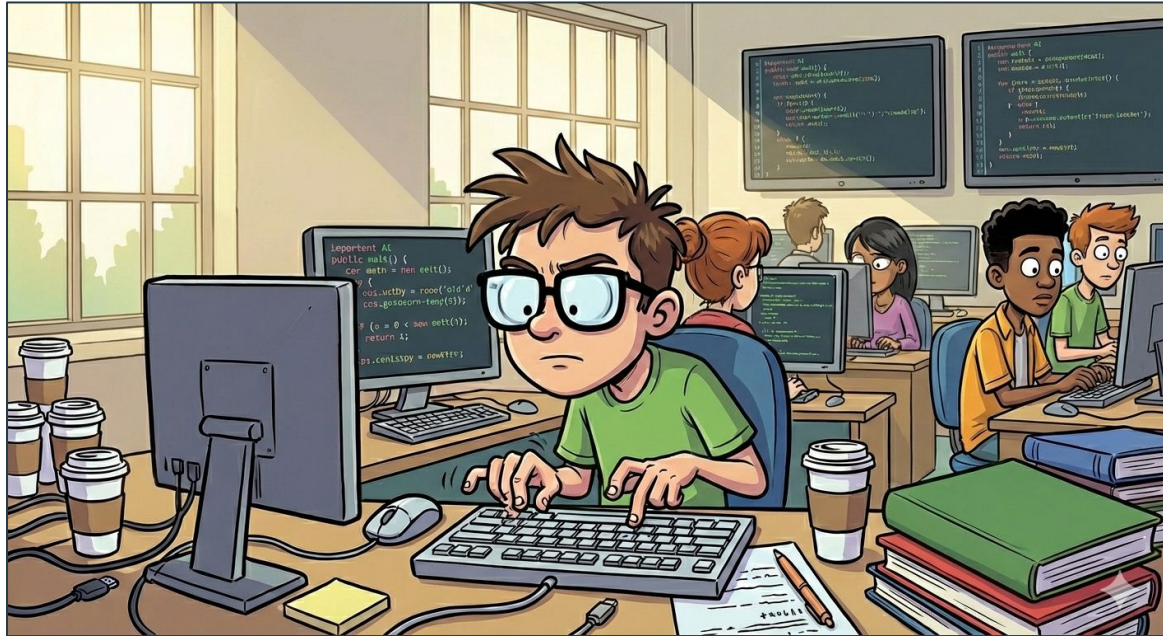
```
# 1. Entrada (Hardware)
# La imagen llega en RGB. (Capa 0 es Rojo)
img_rgb = plt.imread('foto.jpg')

# 2. Traducción (Matemática)
# Creamos una NUEVA matriz con la misma forma.
# Ahora la Capa 0 es el Matiz (Hue).
img_hsv = matplotlib.colors.rgb_to_hsv(img_rgb)

# 3. Lógica (Inteligencia)
# Buscamos cosas rojas SOLO en el canal 0
canal_hue = img_hsv[:, :, 0]
mascara = (canal_hue > 0.9) # Ejemplo para rojo
```

Pasamos a la Práctica

Realiza la Actividad 2: Diagnóstico y Color



De Operaciones Puntuales a Vecindad

Hasta ahora (Puntuales):

- Mirábamos el píxel $P(x, y)$ de forma aislada.
- Si queríamos más brillo, sumábamos 20. Al píxel no le importaba si su vecino era blanco o negro.
- *Problema:* Si el píxel es **ruido** (un punto blanco erróneo), al darle brillo solo hacemos que el error brille más.

Ahora (Vecindad):

- Filosofía: "Un píxel no es nadie sin su contexto".
- Para calcular el nuevo valor de un píxel, consultamos a sus vecinos (normalmente una matriz de 3×3 o 5×5).
- Esto permite: Suavizar, detectar bordes y eliminar errores.

¿Qué es una Convolución?

Es el motor matemático de Photoshop, Instagram y las Redes Neuronales (CNN).

El Mecanismo:

1. Imagina una ventana pequeña (llamada **Kernel** o Máscara) de números.
2. La deslizamos sobre la imagen original, píxel a píxel.
3. En cada posición, **multiplicamos** los valores del Kernel por los píxeles que tiene debajo y **sumamos** el resultado.
4. Esa suma es el **nuevo valor** del píxel central.

Metáfora: Es como pasar un escáner de mano sobre un documento. Lo que lee el escáner depende de la lente (el Kernel) que le pongas.

LA CONVOLUCIÓN: VENTANA DESLIZANTE

120	0	1	120	135	136	200	200
-2	0	2	120	135	170	185	130
-1	0	1	200	120	210	180	175
135	130	135	200	200	160	170	175
130	175	130	115	120	135	125	170
145	170	120	130	145	120	140	165
190	145	190	135	160	175	170	200
130	135	200	130	200	120	135	200

IMAGEN ORIGINAL
(MATRIZ GRANDE)

KERNEL
(MÁSCARA / FILTRO)

IMAGEN RESULTANTE
(MAPA DE CARACTERÍSTICAS)

Cálculo:
 $(-1 \times 120) + (0 \times 135) + \dots$
= RESULTADO

	160			

IMAGEN RESULTANTE
(MAPA DE CARACTERÍSTICAS) ✨

El Poder está en la Matriz

La operación matemática (multiplicar y sumar) siempre es la misma. Lo único que cambia son los números dentro de la ventanita (Kernel).

- **Tamaño:**

- Suelen ser impares (3×3 , 5×5 , 7×7) para tener un píxel central definido.
- Cuanto más grande, más fuerte es el efecto (y más lento el cálculo).

- **La Regla de Oro:**

- Si los números suman **1**: La imagen mantiene su brillo (Filtros de Suavizado).
- Si los números suman **0**: La imagen se vuelve negra, solo quedan los cambios (Filtros de Bordes).

Suavizado y Reducción de Ruido

¿Para qué queremos emborronar una foto a propósito? No es para que se vea mal, es para **eliminar lo que sobra**.

- **El Filtro de Media (Mean Blur):**
 - Calcula el promedio de los 9 vecinos.
 - **Efecto:** Democracia. Si un píxel es un error (blanco puro) rodeado de negro, el promedio lo apagará.
 - **Defecto:** Los bordes nítidos también se pierden. La imagen parece vista "sin gafas".
- **Aplicación:** Pre-procesamiento. Antes de analizar una imagen, la limpiamos de "polvo" digital.

Calculando a mano (La Democracia)

Imagina esta vecindad 3×3 de una imagen (zona oscura con un punto de ruido):

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{250} & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

El 250 es un píxel defectuoso (ruido).

Aplicamos Kernel de Media (todos valen $1/9$):

$$NuevoValor = \frac{10 + 10 + 10 + 10 + 250 + 10 + 10 + 10 + 10}{9}$$

$$NuevoValor = \frac{330}{9} \approx 36$$

Resultado: El 250 (blanco) ha desaparecido y se ha convertido en un 36 (gris oscuro). ¡Hemos arreglado el píxel gracias a sus vecinos!

Detección de Bordes (Edges)

Aquí buscamos lo contrario: **Ignorar las zonas lisas y resaltar los cambios.**

- **¿Qué es un borde?** Matemáticamente, es un cambio brusco de valor (de negro a blanco o viceversa).
- **¿Cómo lo detectamos?** Restando vecinos.
 - Si Vecino A (50) - Vecino B (50) = 0 → Zona lisa (Negro).
 - Si Vecino A (200) - Vecino B (10) = 190 → Borde fuerte (Blanco).

Sobel - El Rey de los Bordes

Utiliza dos Kernels distintos para calcular la "derivada" (el cambio) de la imagen.

Sobel X (Detecta cambios horizontales):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Fíjate: Suma 0. La izquierda es negativa, la derecha positiva.

- Si a la izquierda hay negro (0) y a la derecha blanco (100) → El resultado es positivo (Borde).
- Si todo es igual → El resultado es 0 (Negro).

¿Qué pasa en las esquinas?

Cuando el centro del Kernel está en el borde de la imagen, parte del Kernel "cuelga" en el vacío. No tenemos vecinos.

Soluciones:

1. **Ignorar:** La imagen resultante es un poco más pequeña (se recorta el marco).
2. **Padding (Relleno):** Inventarnos pixeles alrededor.
 - *Zero Padding:* Rellenar con negro (0).
 - *Replicate:* Copiar el último píxel válido (estirar el borde).

Tipos de Kernels I

Tipo de Filtro	Aspecto del Kernel	Efecto Visual	¿Por qué?
Identidad	Un 1 en el centro, 0 en el resto. <pre> 0 0 0 0 1 0 0 0 0 </pre>	La imagen se queda igual.	Al ser todo 0 menos el central, el resultado del cálculo da el mismo valor que el valor central. No cambia ningún pixel.
Suavizado (Blur)	La suma debe ser 1. En el ejemplo, se divide por la cantidad de celdas que haya en el kernel (media) para que se respete el valor de la suma. <pre>1 1 1 1 - X 1 1 1 9 1 1 1 </pre>	Imagen borrosa.	Para que un filtro sea de suavizado (blur) y no altere el brillo de la imagen, la SUMA de todos los números de la matriz debe ser exactamente 1. • Si la suma es > 1: La imagen resultante saldrá más brillante (se "quema"). • Si la suma es < 1: La imagen resultante saldrá más oscura. • Si la suma es 0: Detectarás bordes (la imagen se vuelve negra y solo quedan las siluetas).

Tipos de Kernels II

Tipo de Filtro	Aspecto del Kernel	Efecto Visual	¿Por qué?
Bordes (Sobel)	La mitad de los números son negativos y la otra mitad positivos, la suma de todos ellos tienen que ser de 0. El lugar donde se pongan los positivos y negativos influye en si detecta los bordes del eje X o Y. Eje X: Eje Y: -1 0 1 -1 -2 -1 -2 0 2 0 0 0 -1 0 1 1 2 1 <td>Pone el fondo negro y marca las líneas blancas.</td> <td> Sobel es un detector de aproximación de bordes, calcula un aproximado de la gradiente de la imagen. Marca como negro las zonas “planas” y las blancas como “bordes fuertes”. </td>	Pone el fondo negro y marca las líneas blancas.	Sobel es un detector de aproximación de bordes, calcula un aproximado de la gradiente de la imagen. Marca como negro las zonas “planas” y las blancas como “bordes fuertes”.
Enfoque (Sharpen)	La suma total es 1, el centro debe ser positivo y los vecinos deben ser negativos. 0 -1 0 -1 5 -1 0 -1 0	Imagen crujiente.	Exageras la diferencia entre el centro y los vecinos.

1. IDENTIDAD (Original)



2. BLUR (Suavizado)



3. SOBEL (Bordes)

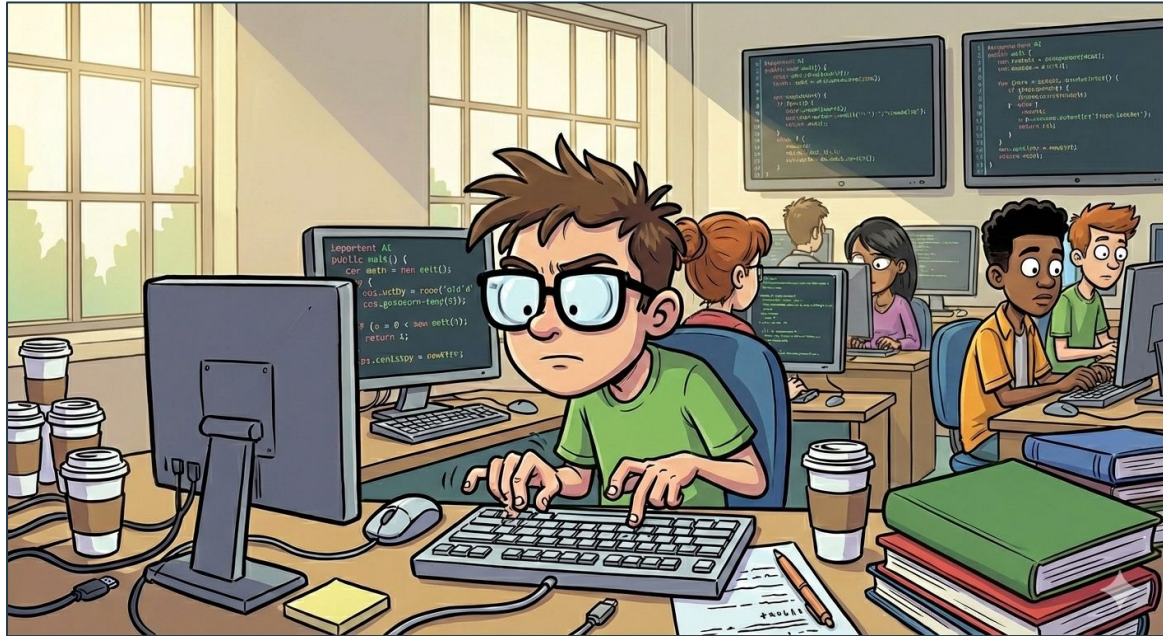


4. SHARPEN (Enfoque)



Pasamos a la Práctica

Realiza la Actividad 3: Filtros y Convoluciones



Introducción a OpenCV y Espacios de Color

De la programación manual a la Visión Artificial Profesional.



PROGRAMACIÓN MANUAL (Bucles for)



Manipulación artesanal de matrices

VISIÓN ARTIFICIAL PROFESIONAL (OpenCV)



Operaciones matriciales optimizadas

¿Cuándo usar OpenCV además de NumPy?

NumPy

- Biblioteca fundamental para cálculo numérico y manipulación de arrays.
- Optimizada en C y apoyada en BLAS/LAPACK.
- Ideal para operaciones matemáticas generales.

OpenCV

- Biblioteca especializada en visión por computador.
- Implementada en C/C++ con optimizaciones SIMD (SSE/AVX).
- Incluye miles de algoritmos listos para usar.

Propiedades

Velocidad

Ambas bibliotecas ejecutan la mayor parte del trabajo en código nativo. OpenCV suele ser más eficiente para operaciones típicas de imagen y vídeo.

Nivel de abstracción

- Con NumPy: las transformaciones geométricas y filtros no están disponibles directamente.
- Con OpenCV: funciones de alto nivel como `cv2.warpAffine()`, `cv2.resize()`, `cv2.GaussianBlur()`.

Ecosistema

OpenCV proporciona detección de objetos, reconocimiento facial, stitching, tracking y módulos de deep learning.

Idea clave

NumPy y OpenCV **no compiten**: se complementan.

Python actúa como interfaz; el rendimiento real proviene del código C/C++.

Gestión de Archivos y Visualización (I/O)

Lectura: `img = cv2.imread('ruta/foto.jpg')`

- Importante: Si la ruta está mal, no da error, devuelve None. ¡Comprobad siempre!

Escritura: `cv2.imwrite('salida.png', imagen_procesada)`

Visualización (HighGUI):

- `cv2.imshow('Ventana', img)`
- `cv2.waitKey(0)` → Bloquea la ejecución hasta pulsar una tecla. **Fundamental** para que la ventana no se cierre al instante.
- `cv2.destroyAllWindows()`

La "Trampa" del BGR

El Conflicto:

- El mundo (Pantallas, Matplotlib, Web) usa **RGB**.
- OpenCV usa **BGR** (Azul-Verde-Rojo) por razones históricas.

Consecuencias:

- Si visualizas una imagen cargada por OpenCV en Matplotlib, los rojos se ven azules (y viceversa).

Solución: Conversión de Espacio de Color.

```
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

Operaciones Geométricas Esenciales

Redimensionar (Resize):

- `cv2.resize(img, (ancho, alto))`
- Útil para normalizar tamaños de datasets.

Recortar (Crop):

- ¡No hay función `cv2.crop`! Usamos **Slicing de NumPy**.
- `recorte = imagen[y_inicio:y_fin, x_inicio:x_fin]`

Voltear (Flip):

- `cv2.flip(img, codigo)` (0=Vertical, 1=Horizontal, -1=Ambos).

Segmentación por Color (HSV)

El Objetivo: No queremos recortar la imagen todavía, solo queremos preguntarle a cada píxel: *"¿Eres del color que busco?"*.

La Herramienta: `cv2.inRange(imagen_hsv, limite_inferior, limite_superior)`.

La Lógica:

- La función recorre la matriz píxel a píxel.
- Si el valor está dentro del rango → Escribe un **255 (Blanco)**.
- Si el valor está fuera → Escribe un **0 (Negro)**.

El Resultado:

- Obtenemos una **Máscara Binaria** (uint8, 1 solo canal).
- No tiene color. Es solo un mapa topográfico de "zonas de interés".

Operaciones Bitwise (El "Estarcido")

El Problema: Tenemos la foto original (con fondo) y la máscara (blanco/negro). ¿Cómo las fusionamos?

La Solución: Aritmética Lógica (Bitwise AND).

La Fórmula:

```
resultado = cv2.bitwise_and(src1=original, src2=original, mask=mascara)
```

Funcionamiento Interno:

1. OpenCV mira el píxel de la máscara.
 2. ¿Es 0 (Negro)? → El resultado es Negro (apaga el píxel).
 3. ¿Es > 0 (Blanco)? → Deja pasar el color de la imagen original.
- *Es como poner una plantilla de cartón sobre la foto y pintar encima.*

Calibración Interactiva (HighGUI)

El Desafío: La luz cambia. Un rojo "teórico" ($H=0$) puede verse como $H=5$ o $H=175$ dependiendo de la bombilla de la habitación. Adivinar los números del `inRange` es ineficiente.

La Solución Profesional: Trackbars (Barras de deslizamiento).

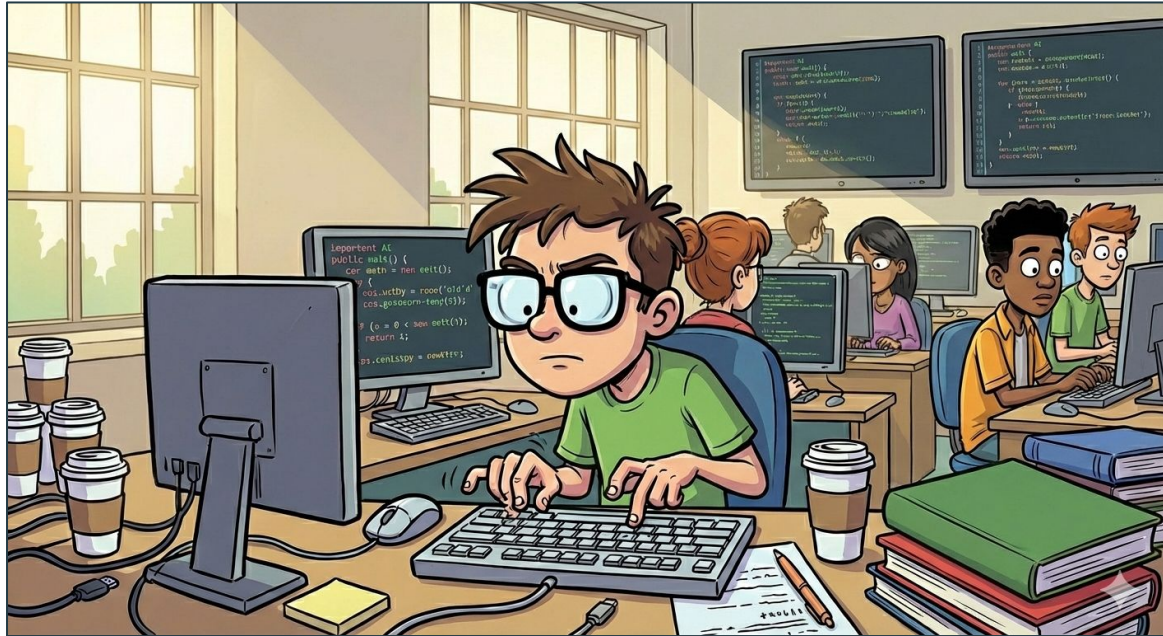
Algoritmo de Calibración:

1. Crear ventana y 6 barras (`H_min`, `H_max`, `S_min`, `S_max`, `V_min`, `V_max`).
2. **Bucle Infinito (While True):**
 - Leer valores actuales de las barras (`cv2.getTrackbarPos`).
 - Generar máscara con esos valores al instante.
 - Mostrar máscara y resultado.
 - Esperar 1ms (`cv2.waitKey(1)`).

Resultado: Un sintonizador visual en tiempo real.

Pasamos a la Práctica

Realiza la Actividad 4: Introducción a OpenCV



El Problema del Umbral Fijo (threshold)

La limitación del valor "127":

- Hasta ahora elegíamos un valor manual (ej: 127) para separar blanco y negro.
- **Escenario A (Imagen Oscura):** Si la luz baja, el objeto gris se vuelve negro. El umbral 127 es *demasiado alto*.
- **Escenario B (Imagen Sobreexpuesta):** Si hay mucha luz, el fondo se vuelve blanco. El umbral 127 es *demasiado bajo*.

Conclusión:

- La iluminación en el mundo real nunca es constante.
- Necesitamos un algoritmo que calcule el umbral "perfecto" analizando cada imagen individualmente.

El Histograma: El Mapa de Intensidades

Recordemos el histograma.

- Es un gráfico de barras que cuenta cuántos píxeles hay de cada tono de gris.
- **Eje X:** Intensidad (0 a 255).
- **Eje Y:** Cantidad de píxeles.

La Distribución Bimodal:

- Para poder separar un objeto del fondo, necesitamos ver dos "montañas" claras en el gráfico:
 1. El pico de los píxeles del **Fondo**.
 2. El pico de los píxeles del **Objeto**.
- **El objetivo:** Encontrar el punto más profundo del "valle" entre esas dos montañas. Ese es el umbral de corte ideal.

El Método de Otsu (Binarización Inteligente)

¿Cómo funciona?

- Es un algoritmo que calcula automáticamente el umbral óptimo iterando por todos los valores posibles.

La Lógica Matemática:

- Busca minimizar la **Varianza Intra-clase**.
- Traducción: Busca el número que hace que el grupo de "blancos" y el grupo de "negros" sean lo más compactos y definidos posible, reduciendo el ruido.

Implementación en OpenCV:

- Se pasa el valor 0 como umbral inicial.
- Se añade la bandera `cv2.THRESH_OTSU`.
- La función devuelve el valor calculado (ej: 142) que ha usado para cortar.

Requisito: Solo funciona bien si el histograma es **Bimodal** (dos picos claros).

Introducción a la Morfología Matemática

El Problema:

- Tras binarizar (incluso con Otsu), la imagen suele tener "ruido":
 - Puntos blancos sueltos en el fondo (polvo).
 - Puntos negros dentro del objeto (huecos).

La Solución:

- Operaciones basadas en la **FORMA** de los objetos, no en su color.

La Herramienta: El Kernel (Elemento Estructurante):

- Una pequeña matriz (cuadrada o circular, ej: 3x3 o 5x5) que se desliza sobre la imagen como un escáner.
- Toma decisiones sobre cada píxel basándose en sus vecinos.

Operación 1: Erosión (Erode) - "Adelgazar"

Concepto:

- Reduce el tamaño de las zonas blancas. "Come" los bordes de los objetos.

Regla Lógica (AND):

- El píxel central se mantiene **BLANCO** solo si **TODOS** los píxeles bajo el Kernel son blancos.
- Si hay un solo píxel negro vecino $\rightarrow$ El centro se vuelve negro.

Usos:

1. Eliminar puntos blancos pequeños (ruido) del fondo.
2. Separar dos objetos que están unidos por una línea fina.
3. Despegar objetos que tocan los bordes de la imagen.

Operación 2: Dilatación (Dilate) - "Engordar"

Concepto:

- Aumenta el tamaño de las zonas blancas. Expande los bordes.

Regla Lógica (OR):

- El píxel central se convierte en **BLANCO** si **AL MENOS UNO** de los píxeles bajo el Kernel es blanco.
- Solo se queda negro si absolutamente todos los vecinos son negros.

Usos:

1. Rellenar pequeños agujeros negros dentro de un objeto.
2. Unir partes de un objeto que están rotas (ej: texto discontinuo).
3. Hacer los objetos más visibles y sólidos.

Apertura (Opening) = Erosión + Dilatación

Objetivo:

- Eliminar el ruido del fondo (puntos blancos) **SIN** cambiar el tamaño final del objeto principal.

El Proceso:

1. **Paso 1 (Erosión):** El ruido pequeño desaparece por completo. El objeto principal se encoge.
2. **Paso 2 (Dilatación):** El objeto principal recupera su tamaño original. El ruido, como ya fue eliminado, no vuelve.

Analogía: "Abrir la puerta para sacar la basura y volverla a cerrar".

Cierre (Closing) = Dilatación + Erosión

Objetivo:

- Cerrar agujeros internos o puntos negros dentro del objeto **SIN** engordar el objeto final.

El Proceso:

1. **Paso 1 (Dilatación):** Los agujeros pequeños se cierran al expandirse el blanco. El objeto se hace más grande temporalmente.
2. **Paso 2 (Erosión):** El objeto vuelve a su tamaño original ("lijar bordes"), pero los agujeros internos permanecen cerrados.

Analogía: "Masillar grietas en una pared y luego lijar para que quede liso".

Resumen: Guía de Selección de Operaciones

Síntoma en la Imagen	Operación Recomendada	Efecto
Iluminación variable	THRESH_OTSU	Calcula el umbral ideal.
Puntos blancos (ruido) en fondo	MORPH_OPEN (Apertura)	Limpia el fondo.
Puntos negros dentro del objeto	MORPH_CLOSE (Cierre)	Rellena el objeto (Sólido).
Objeto roto / Texto discontinuo	dilate (Dilatación)	Conecta fragmentos.
Objetos pegados por hilos finos	erode (Erosión)	Separa objetos.

Nota: El tamaño del Kernel (3x3, 5x5, 7x7) determina la intensidad de la operación.

Pipeline de Procesamiento

1. **Input:** Imagen Color.
2. **Grayscale** (Elimina color).
3. **Blur** (Suaviza histograma).
4. **Threshold Otsu** (Separa Blanco/Negro).
5. **Morphology** (Limpia ruido).
6. **Output:** Máscara Binaria Perfecta.

